PPTS 5

Parte 1 Eva 3 Primavera 2026

Unidad 2

Docente: Boris Astorga
Período: Primavera 2026

1 . Tecnologías del lado del servidor | Horas de la Unidad: 16 | Horas Presenciales: 16 | Horas Online: 0

APRENDIZAJES ESPERADOS	CRITERIOS DE EVALUACIÓN	CONTENIDOS MÍNIMOS OBLIGATORIOS
1.1 Utiliza tecnologías del lado del servidor para resolver problemáticas planteadas, de acuerdo con la sintaxis del lenguaje y el uso de herramientas de inteligencia artificial como apoyo para mejorar la eficiencia del desarrollo.	1.1.1 Identifica variables y operaciones del lenguaje, según requerimiento. 1.1.2 Codifica instrucciones, estructuras y operadores, según requerimientos. 1.1.3 Codifica instrucciones utilizando paquetes externos, según requerimientos. 1.1.4 Implementa una aplicación sencilla en Django, según los requerimientos establecidos y las recomendaciones generadas por herramientas de inteligencia artificial.	Tecnologías del lado del servidor Protocolos, servicios de hosting y dominios. Modelo Vista Controlador (MVC) Django Framework, Instalación y Configuración Django Models, Vistas y Templates Utilización de IA como apoyo a diseño de interfaces y generación de datos de prueba

2 . Framework back end | Horas de la Unidad: 36 | Horas Presenciales: 36 | Horas Online: 0

APRENDIZAJES ESPERADOS	CRITERIOS DE EVALUACIÓN	CONTENIDOS MÍNIMOS OBLIGATORIOS
2.1 Codifica aplicaciones web para proponer soluciones tentativas a una problemática, utilizando un framework del lado del servidor y herramientas de inteligencia artificial como apoyo al desarrollo.	2.1.1 Configura la conexión a una base de datos, según requerimiento. 2.1.2 Utiliza el administrador de Django, de acuerdo a los requerimientos. 2.1.3 Codifica funciones que realicen operaciones CRUD, sobre una base de datos según requerimiento. 2.1.4 Codifica una aplicación back end con acceso a base de datos y seguridad, según los requerimientos del problema planteado y orientaciones generadas por herramientas de inteligencia artificial.	Conexión a una base de datos a través del Framework. Django Admin. Django ORM. Operaciones CRUD a través del Framework. Apoyo de IA para la estructuración del código

Unidades, aprendizajes esperados y criterios

Unidad (horas)	Aprendizaje esperado	Criterios de evaluación
1 .Tecnologías del lado del servidor	1.1 Utiliza tecnologías del lado del servidor para resolver problemáticas planteadas, de acuerdo con la sintaxis del lenguaje y el uso de herramientas de inteligencia artificial como apoyo para mejorar la eficiencia del desarrollo.	1.1.1 Identifica variables y operaciones del lenguaje, según requerimiento. 1.1.2 Codifica instrucciones, estructuras y operadores, según requerimientos. 1.1.3 Codifica instrucciones utilizando paquetes externos, según requerimientos
2. Framework back end	2.1 Codifica aplicaciones web para proponer soluciones tentativas a una problemática, utilizando un framework del lado del servidor y herramientas de inteligencia artificial como apoyo al desarrollo.	2.1.1 Configura la conexión a una base de datos, según requerimiento. 2.1.2 Utiliza el administrador de Django, de acuerdo a los requerimientos. 2.1.3 Codifica funciones que realicen operaciones CRUD, sobre una base de datos. 2.1.4 Codifica una aplicación back end con acceso a base de datos y seguridad, según los requerimientos del problema planteado y orientaciones generadas por herramientas de inteligencia artificial.
3. API restful con integración a base de datos	3.1 Codifica una API RESTful para conectarse con otras aplicaciones, utilizando autenticación JWT y herramientas de inteligencia artificial como apoyo al desarrollo.	3.1.1 Configura Django Rest framework, según la documentación oficial. 3.1.2 Codifica instrucciones de autenticación, de acuerdo con los requerimientos y considerando recomendaciones de seguridad generadas por herramientas de inteligencia artificial.

Back End en una mirada

AJUSTES AL DESCRIPTOR

Asignatura practica del area de Tecnologias de Informacion y Ciberseguridad, en modalidad presencial.

Al finalizar, el estudiante sera capaz de construir una aplicacion web y una API RESTful, integrando herramientas de IA para apoyar la codificacion, depuracion, prototipado y validacion de soluciones.

Competencia transversal: Resolucion de Problemas, Innovacion y Emprendimiento, integrando tecnologias digitales propias de la industria 4.0.

72

horas de laboratorio

100% practica

3

unidades de aprendizaje

Django, CRUD y API restful

4

evaluaciones sumativas

Individuales, escala de apreciacion

4

nivel MCTP

UCL: desarrollo en entorno servidor

MCTP: Marco de Cualificaciones Tecnico-Profesional · UCL: Unidad de Competencia Laboral (Desarrollar elementos software en el entorno servidor, UC0492_3, ajustada al contexto nacional).

Unidades y evaluacion sumativa

AJUSTES AL DESCRIPTOR

UNIDAD 1

16 hrs

Tecnologias del lado del servidor

MVC, Django (models, vistas y templates) e IA para interfaces y datos de prueba.

ES1 · 15%

Archivo de programa en Django

UNIDAD 2

36 hrs

Framework back end

Base de datos, Django Admin, ORM, CRUD y apoyo de IA para estructurar el codigo.

ES2 · 25% + ES3 · 30%

App Django Admin / App CRUD con sesiones

UNIDAD 3

20 hrs

API restful con base de datos

REST, DRF, JSON, JWT y buenas practicas de API con recomendaciones de IA.

ES4 · 30%

API RestFul funcional

Todas las evaluaciones son individuales, con escala de apreciacion. Estrategia didactica: ABP con clase expositiva y practicas de laboratorio.

Evaluación 3
Semana 27-28 Octubre 2026

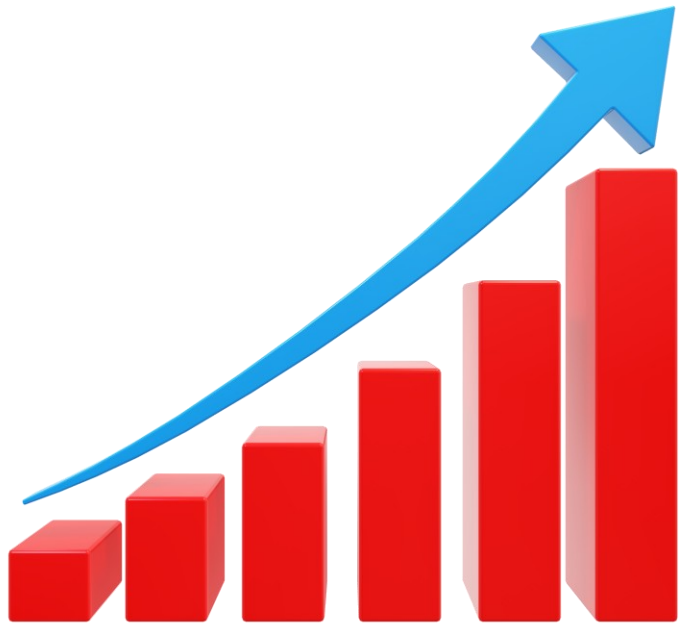
septiembre-26

L	M	X	J	V	S	D
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30				

noviembre-26

L	M	X	J	V	S	D
						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30						

lunes	martes	miércoles	jueves	viernes	sábado	domingo
28	29	30	1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	1
2	3	Notas				



Análisis integral del proyecto CallCenterGamer

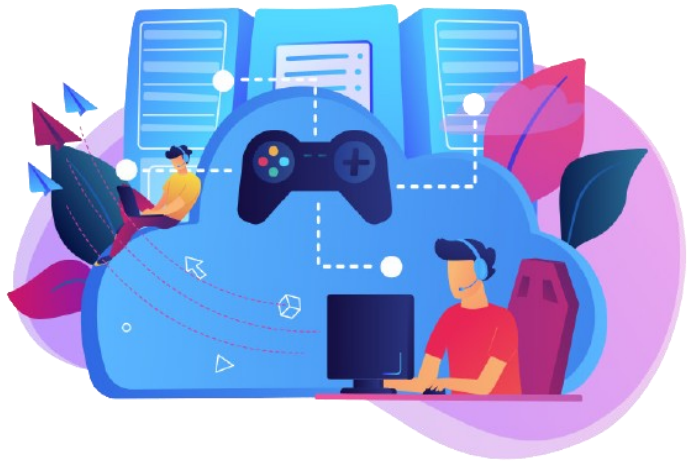
El proyecto plantea una plataforma de soporte técnico para jugadores construida con Django, con administración de usuarios, conversaciones entre clientes y técnicos, y algunas funciones denominadas "IA".

1.1 Propósito del sistema

CallCenterGamer es una aplicación web de mesa de ayuda especializada en videojuegos y hardware gaming.

Su objetivo es permitir que:

- Un visitante se registre o inicie sesión.
- Un cliente cree consultas técnicas.
- Clientes, técnicos y administradores participen en conversaciones.
- Técnicos y administradores revisen consultas.
- Administradores gestionen usuarios y preguntas.
- El sistema clasifique consultas y genere recomendaciones usando IA.



Material Elaborado por el docente Boris Astorga

Análisis integral del proyecto CallCenterGamer

1.1 Propósito del sistema



En términos funcionales, el proyecto se encuentra entre:

- Un **sistema de tickets básico**.
- Un **chat de soporte técnico**.
- Un **panel administrativo**.
- Un experimento de **asistencia y analítica con modelos de lenguaje**.

Material Elaborado por el docente Boris Astorga

2. Arquitectura identificada

La aplicación utiliza una arquitectura monolítica basada en el patrón **MVT de Django**:

Evaluación arquitectónica

La organización general es adecuada para un proyecto pequeño o académico. Las responsabilidades están divididas en dos aplicaciones:

- `online`: operación principal del sistema.
- `administracion`: gestión administrativa.

Sin embargo, la lógica de IA, presentación, autorización y negocio está demasiado concentrada en las vistas. Si el sistema creciera, sería conveniente introducir una capa de servicios.

3. Modelo de dominio

3.1 Entidades

El sistema contiene tres entidades principales:

Usuario

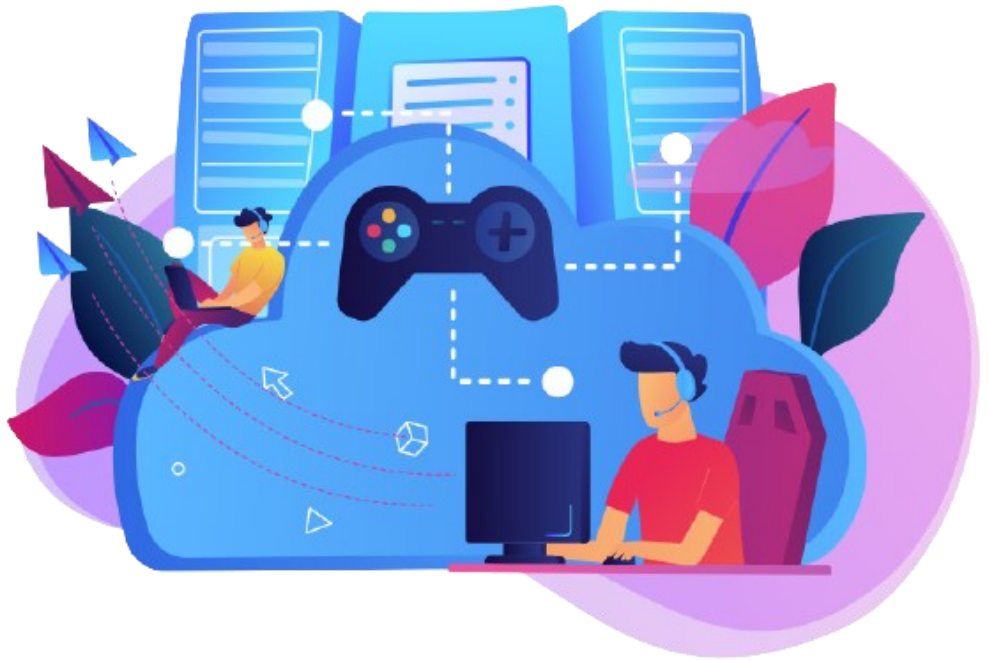
Representa a cualquier persona que usa la plataforma.

Pregunta

Representa una consulta o ticket.

Respuesta

Representa cada mensaje enviado dentro de una consulta.



3.2 Relaciones

Conceptualmente:

- Un usuario puede crear muchas preguntas.
- Una pregunta pertenece a un usuario.
- Una pregunta puede recibir muchas respuestas.
- Cada respuesta tiene un autor.

Material Elaborado por el docente Boris Astorga

3.3 Observación conceptual importante

Aunque el modelo se denomina `Preguntas`, funcionalmente representa un **ticket de soporte**. Las respuestas representan mensajes de conversación.

Actualmente faltan propiedades esenciales de un sistema de tickets:

- Estado: abierto, en proceso, resuelto o cerrado.
- Prioridad.
- Técnico asignado.
- Categoría persistida.
- Fecha de resolución.
- Tiempo real de primera respuesta.
- Etiquetas.
- Historial de cambios.
- Archivos adjuntos.



Material Elaborado por el docente Boris Astorga

4. Roles y permisos

El sistema maneja tres perfiles usando dos campos booleanos:

Perfil	admin	técnico
Cliente	False	False
Técnico	False	True
Administrador	True	normalmente True o False



Capacidades actuales

Cliente

- Crear preguntas.
- Ver sus propias preguntas.
- Participar en sus propias conversaciones.
- Consultar su historial.

Técnico

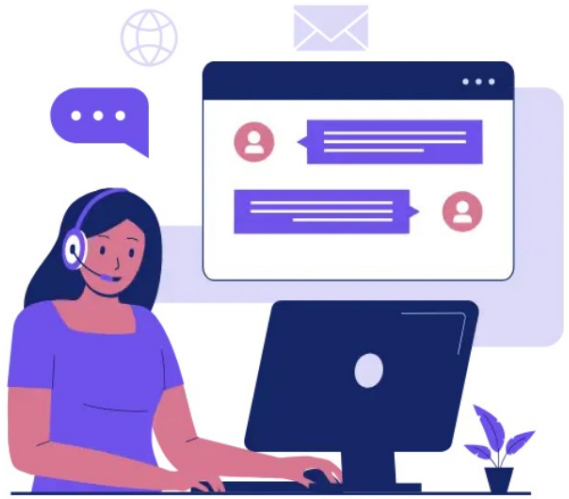
- Acceder a `/Administracion/`.
- Ver conversaciones de otros usuarios.
- Responder consultas.
- No puede gestionar usuarios.
- No puede acceder a Analytics IA.

Administrador

- Acceder al panel administrativo.
- Ver todas las consultas.
- Modificar preguntas.
- Eliminar preguntas.
- Administrar roles y estado de usuarios.
- Eliminar usuarios.
- Consultar analytics.

Evaluación

La autorización principal se aplica en el backend, lo cual es correcto. El proyecto no depende exclusivamente de ocultar botones en los templates.



También existen pruebas para intentos como:

- Suplantar el identificador de administrador.
- Ver conversaciones ajenas.
- Manipular campos ocultos.
- Ejecutar eliminaciones mediante GET.

Eso constituye una fortaleza real del proyecto.

```
if not request.user.admin:
    messages.error(request, "No tienes permisos de administrador.")
    return redirect('/Inicio/' + str(request.user.id))
```

Material Elaborado por el docente Boris Astorga

Limitación del diseño de roles

Usar booleanos independientes permite estados ambiguos:

admin=True, técnico=False
admin=False, técnico=True
admin=True, técnico=True
admin=False, técnico=False

No necesariamente es incorrecto, pero complica las reglas de negocio.



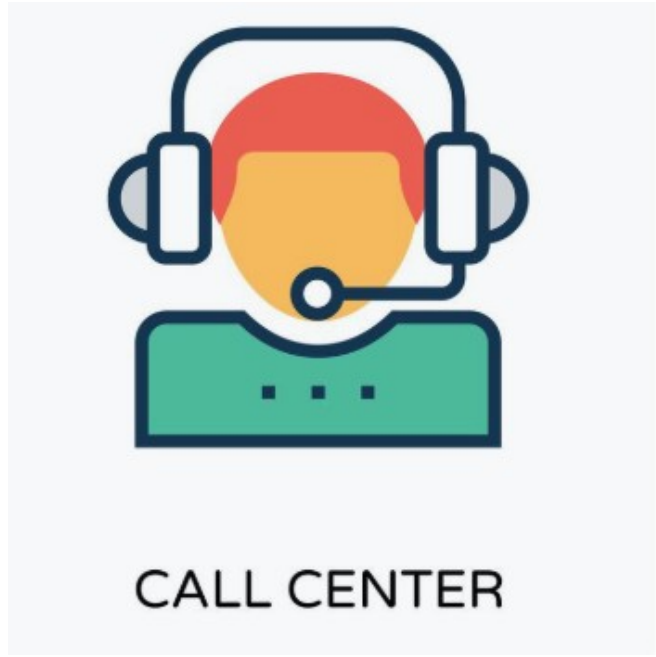

```
rol = models.CharField(
    choices=[
        ("cliente", "Cliente"),
        ("tecnico", "Técnico"),
        ("administrador", "Administrador"),
    ]
)
```

Material Elaborado por el docente Boris Astorga

Otra opción más integrada con Django sería utilizar:



- `Group`
- `Permission`
- `is_staff`
- `is_superuser`



5. Flujos funcionales

5.1 Registro

El usuario completa:

- Nombre de usuario.
- Correo.
- RUT.
- Contraseña.
- Confirmación de contraseña.



El formulario:

- Comprueba que el correo no esté registrado.
- Comprueba que las contraseñas coincidan.
- Exige una longitud mínima.
- Guarda la contraseña mediante `set_password()`.

Aspectos positivos

- La contraseña no se almacena en texto plano.
- Usuario y correo son únicos.
- Django gestiona el hash de contraseña.

Material Elaborado por el docente Boris Astorga

6. Pruebas

Cobertura actual



Administración

- Un cliente no puede borrar preguntas suplantando un ID.
- Borrar preguntas requiere POST.
- Un administrador puede borrar preguntas.
- Eliminar usuarios requiere POST.
- Un administrador no puede eliminarse.
- Un administrador puede eliminar a otro usuario.
- Analytics está disponible para administradores.
- Analytics está bloqueado para técnicos.



Conversaciones

- Un cliente no puede ver conversaciones ajenas.
- Administradores y técnicos pueden verlas.
- Los campos ocultos manipulados no cambian autor ni pregunta.
- Logout requiere POST.
- Usuarios inactivos no pueden iniciar sesión.

Material Elaborado por el docente Boris Astorga

Pruebas faltantes prioritarias



1. Registro con correo duplicado.
2. Registro con usuario duplicado.
3. Validación real del RUT.
4. Creación de pregunta ignorando usuario adulterado.
5. Modificación de preguntas solo por administradores.
6. Técnico no puede modificar o borrar.
7. Administrador no puede desactivarse.
8. No se puede eliminar al último administrador.
9. Navegación del panel administrativo.
10. IA sin clave configurada.
11. IA con tiempo de espera.
12. Cálculo real de analytics.
13. Respuesta con longitud excesiva.
14. Formulario inválido conserva errores.
15. Pregunta inexistente devuelve un comportamiento adecuado.
16. Acceso no autenticado a todas las rutas protegidas.
17. Pruebas de templates para detectar loops y enlaces incorrectos.

Material Elaborado por el docente Boris Astorga

Evaluación de mantenibilidad

Fortalezas

- Separación básica por aplicaciones.
- Uso de forms de Django.
- Modelo de usuario personalizado.
- Pruebas de autorización.
- Configuración mediante variables de entorno.
- Funciones de IA separadas en módulos.
- Uso correcto de POST para operaciones destructivas.



Debilidades

- Vistas demasiado extensas.
- Lógica de IA mezclada con renderizado.
- Excepciones demasiado amplias.
- Código duplicado.
- Métricas ficticias.
- Templates estructuralmente inconsistentes.
- Navegación dependiente de JavaScript.
- Estilos inline.
- Nombres inconsistentes.
- Falta de estados para los tickets.
- IA ejecutada de forma síncrona.
- Dependencias sin versiones reproducibles.

Material Elaborado por el docente Boris Astorga

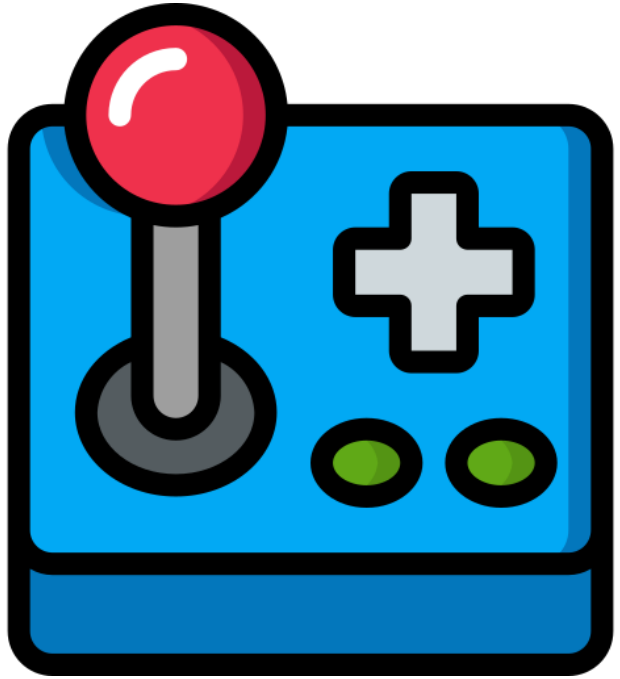
Diagnóstico por prioridad

Prioridad crítica

1. Corregir el loop de `/Administracion/`.
2. Reemplazar `indexAdmin.html` por un panel administrativo real.
3. Verificar que `<`, `>` y los enlaces HTML no estén literalmente dentro del código.
4. Conectar el asistente de `inicio.html` con la vista `Inicio`.
5. Evitar que errores internos de OpenAI se muestren al usuario.

Prioridad alta

6. Fijar versiones compatibles de Django y dependencias.
7. Evitar decenas de llamadas de IA por página.
8. Guardar la categoría en la base de datos.
9. Calcular realmente el tiempo promedio de respuesta.
10. Evitar autodesactivación y autodegradación del administrador.
11. Normalizar la estructura de templates.
12. Validar el RUT en el servidor.
13. Añadir límites de longitud del lado servidor.
14. Definir política de privacidad para uso de IA.



Material Elaborado por el docente Boris Astorga

Prioridad media

15. Eliminar templates duplicados.
16. Mejorar diseño responsivo.
17. Mover CSS inline a archivos.
18. Introducir estados y prioridades del ticket.
19. Usar logging en vez de `print`.
20. Eliminar imports y variables sin uso.
21. Optimizar consultas.

Prioridad baja

22. Normalizar nombres de variables.
23. Corregir títulos y textos.
24. Eliminar comentarios duplicados.
25. Renombrar `requeriments.txt`.



Material Elaborado por el docente Boris Astorga

Roadmap recomendado

Fase 1: estabilización

- Corregir el panel administrativo.
- Eliminar redirecciones JavaScript redundantes.
- Arreglar templates que extienden `header.html`.
- Validar que el código no contenga entidades HTML escapadas.
- Fijar dependencias.
- Ejecutar `manage.py check`.
- Ejecutar migraciones.
- Ejecutar todas las pruebas.

Fase 2: consistencia funcional

- Conectar el asistente del cliente.
- Crear un campo `categoria`.
- Crear un campo `estado`.
- Crear prioridad.
- Calcular tiempo de respuesta real.
- Resolver templates duplicados.
- Proteger la gestión del último administrador.

Material Elaborado por el docente Boris Astorga

Roadmap recomendado

Fase 3: seguridad y privacidad

- No exponer excepciones.
- Incorporar logging.
- Validar RUT en backend.
- Limitar longitudes.
- Definir política de privacidad.
- Aplicar límites de uso de IA.
- Añadir auditoría de acciones administrativas.

Fase 4: rendimiento

- Clasificar preguntas una sola vez.
- Mover tareas de IA a segundo plano.
- Cachear analytics.
- Optimizar relaciones.
- Paginar preguntas y usuarios.

Material Elaborado por el docente Boris Astorga

Roadmap recomendado

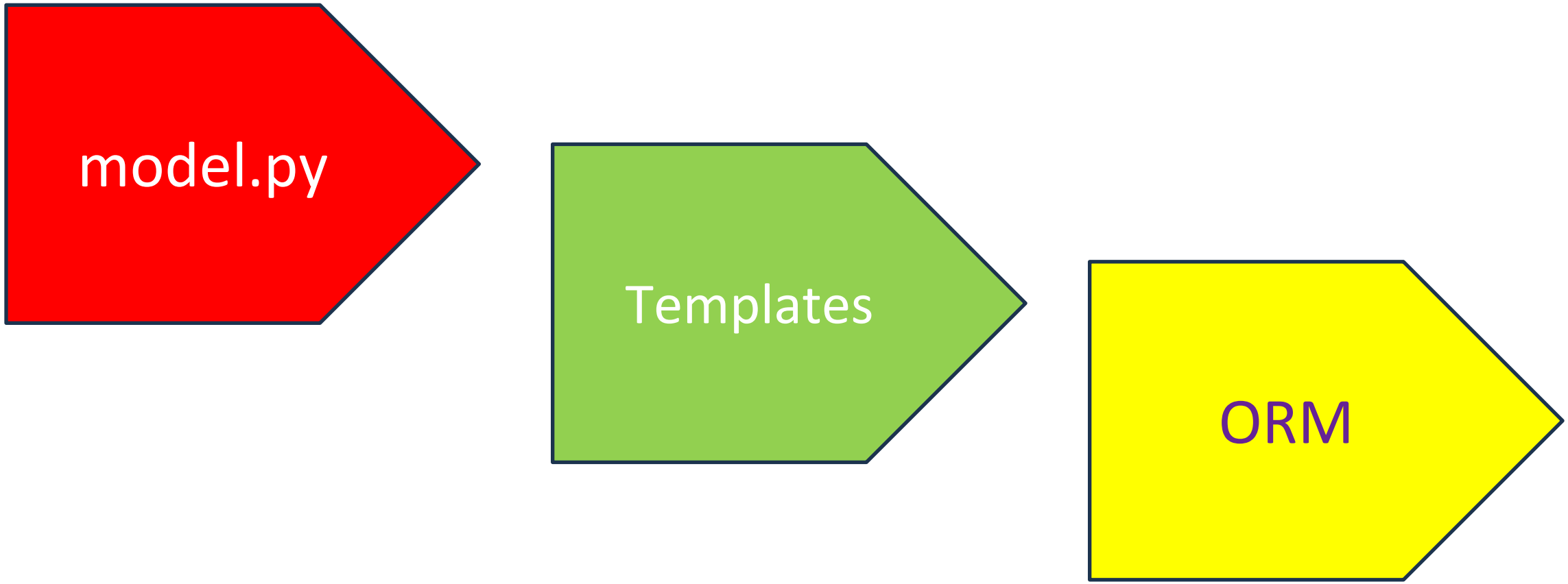
Fase 5: evolución del producto

- Asignación de técnicos.
- Estados de ticket.
- Priorización.
- Cierre y reapertura.
- Encuesta de satisfacción.
- Dashboard real.
- Notificaciones.
- Archivos adjuntos.
- Historial de auditoría.



Material Elaborado por el docente Boris Astorga

Retroalimentación



Material Desarrollado por el Docente Boris Astorga
boris.astorga02@inacapmail.cl
Unidad 2 Back End
Framework Django